ECE3375B: Design Project

Objectives

- The ECE3375 design project was developed by Prof. Ken McIsaac, who taught this course previously.
- He considered most students had the technical skills necessary to complete their **capstone projects** in 4th year, but many were inexperienced in planning these projects.
- This project is intended to give students experience in the design process for an embedded system.
- In this project, you should design **some microcontroller-based system that acquires input and produces output to perform some useful task.**

Please work in **groups of 2 to 4** to complete this project.

Some Example Projects

- Some previous projects:
  1. A wearable device to measure wind chill while skiing.
  2. A system to set individual preferences for water temperature and pressure in a shower.
  3. A RFID-based door lock and light switch.
  4. Automated system to turn on and off room lights based on occupancy.
  5. Motor controller for an electric bicycle.
  6. Home thermostat with multiple remote temperature sensors.
  7. System to lock a bathroom stall and alert custodial staff when the toilet paper runs out.
  8. Arcade game using the mini VGA monitor on the DE10-Standard board.
- You are welcome to choose any of these or come up with your own.

What this project is, and what it is not…

- This project is **not** about making a mechatronic system, or realizing any part of your design in hardware.
- This project is about:
  - Designing an embedded system.
  - Providing a functional description of that system.
  - Developing a plan for implementing the software for that system without necessarily having access to the required hardware.
  - Attempting to implement the software prototype.
  - Revising the software plan as necessary.
  - Documenting your experience.

Deliverables

- A progress report due **March 21th**.
- A final report due **April 4th**.
- A project demonstration will be scheduled with the TAs **during the scheduled project demonstrations.**
- Each of these items is described in detail in the project manual on OWL.
- Lab space and TAs will be available during the regular lab hours the **week of March 24th** for project demonstrations and if you have questions or concerns. (No regular lab is scheduled that week.)

Deliverables

- A progress report is due March 21th. This should cover:
  1. **Problem Definition**: A description of your system.
  2. **Functional Description**: Technical details of the user interface, interactivity, how the system connects to other systems, etc.
  3. **Input/Output Requirements**: The specific data that the system will measure or produce. The specific channels that the system will use for I/O.
  4. **Initial Software Design**: Basic program flow for your system.
  5. **Prototyping Plan**: What parts you can implement given the time and resource constraints, and how you will test them.
- Each of these items is described in detail in the project manual on OWL.

Deliverables

- A final report is due April 4th. This should include the material from the progress report (with any appropriate revisions) and also cover:
  6. **Microcontroller Selection**: Compare at least 2 different microcontrollers and explain which is best for your system.
  7. **Revised Software Design**: Comment if your original design plan needs revision.
  8. **Results from Prototyping**: What worked, what didn't, etc.
  9. **Source Code**: Submit whatever you have.
  10. **Conclusions**: Lessons learned, future plans, etc.
- Each of these items is described in detail in the project manual on OWL.

Deliverables

- A demonstration of whatever you have developed at **the scheduled project demonstrations posted on OWL.**
- To reiterate, this project has a strong focus on **following through the 10 steps outlined above and documenting your progress**.
- This project has only a weak focus on completing all the software for your embedded system.

Software Prototyping

- Embedded systems require both software and hardware development.
- **Invariably, the hardware is always late.**
- This project is about decoupling the hardware design (which you won't do) from the software design.
- The idea is that most hardware can be "faked" as an appropriate set of microcontroller inputs/outputs.
- This is what you need to do to implement your design in software.
  - Use the DE10-Standard as a stand-in for whatever microcontroller system you intend to use.

Example

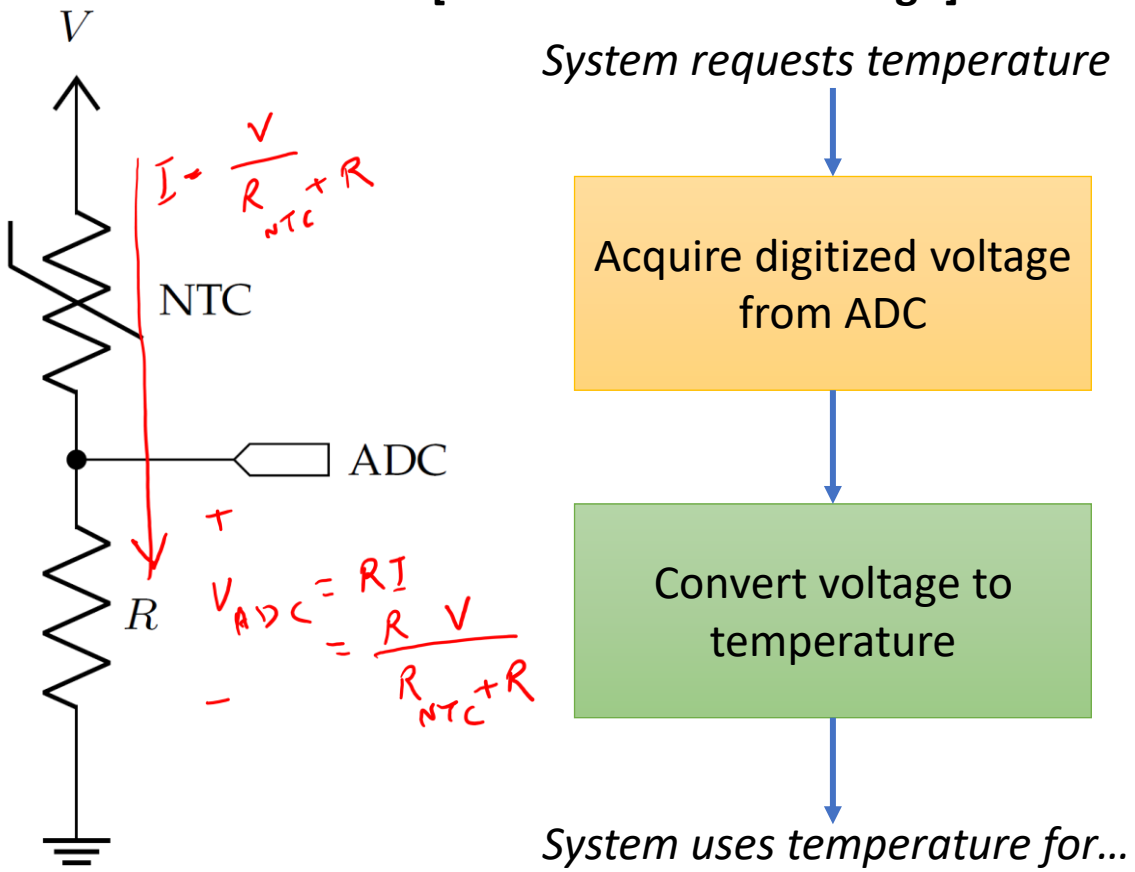
- You are designing an embedded system that will do something useful [**1. Problem Definition**].
- One of the functions of this system is measuring the temperature local to the microcontroller [**2. Functional Description**].
  - This means the temperature can be measured at the microcontroller, or by a peripheral connected by a short wire. No long-range remote sensing is needed.
- The DE10-Standard **does not** have a temperature gauge peripheral built-in.
- Options for temperature sensors include [**3. I/O**]:
  a) Using a NTC thermistor to make an analog measurement.
  b) Using a AS621 chip to make a digital measurement transmitted using I^2C communications.
  c) Using a MAX6576 chip to make a digital measurement transmitted using pulse width modulation (PWM).

Example

- Option (a): NTC (Negative Temperature Coefficient) Thermistor voltage divider
  - The voltage drop across a NTC thermistor is related to the temperature.
  - This is an analog measurement, so it needs to connect to a ADC.

V_{ADC}	T
1	25°
1.1	25.5
1.15	26°

[4. Initial Software Design]



[5. Prototyping Plan]

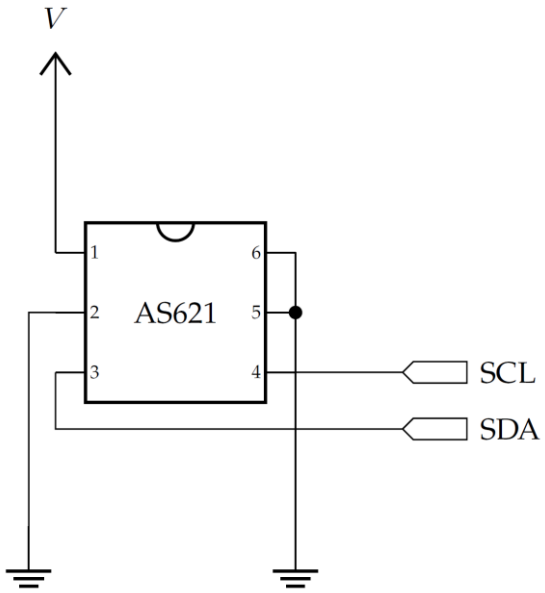
- **Write function to initialize ADC conversion, wait for completion, and return result**.
- Test this function using arbitrary ADC input.
- **Write function to convert 12-bit ADC measurement to temperature**.
- Math is tricky for a microcontroller, maybe make a look-up table mapping ADC voltage to temperature?
- Test this function entirely in software, compare to by-hand calculations.

$$V_{NTC} = V - v_{ADC} = \left(\frac{R_{NTC}}{R + R_{NTC}}\right)V$$

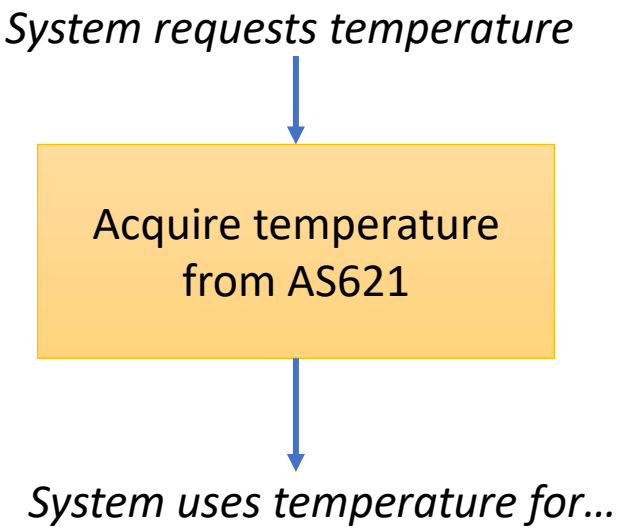
$$R_{NTC} = R_0 \exp\left(\frac{\beta}{T} - \frac{\beta}{T_0}\right) \qquad R_{NTC}\big|_{T=T_0} = R_0$$

Example

- Option (b): AS621 chip
  - AS621 chip measures temperature in °C as a 16-bit signed 2's complement number.
  - AS621 communicates with the microcontroller using I^2C communications protocol.



[4. Initial Software Design]



[5. Prototyping Plan]

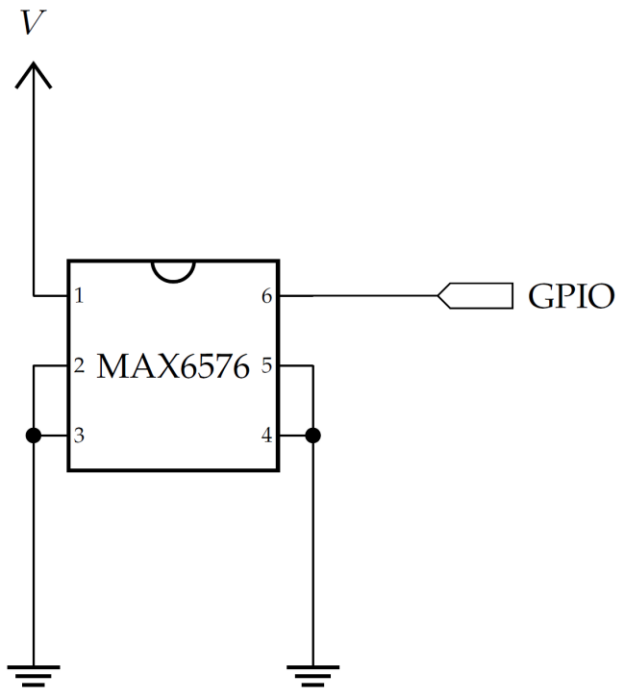
- Must **write code to initialize AS621 address** into microcontroller I^2C network.
- Hard to test without actual hardware.

- **Write function to request and acquire temperature from AS621.**
- Test this function using arbitrary data from anything that is already available on I^2C.
- The DE10-Standard does have I^2C and has at least one built-in peripheral that uses it, so this can act as a stand-in for the AS621.

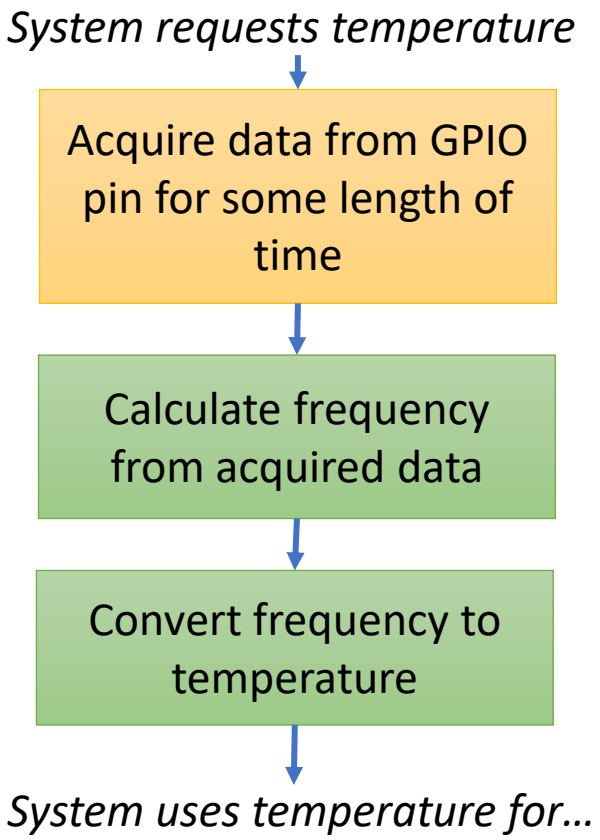
- No data conversion is necessary.

Example

- Option (c): MAX6576 Pulse-Width Modulation
  - The MAX6576 outputs a square wave with frequency/period (τ) related to the temperature T.
  - Can read this data using GPIO.



[4. Initial Software Design]



[5. Prototyping Plan]

- **Write function to time interval until GPIO changes state.**
- Save result to memory.
- Test this function using any available GPIO (even switches/buttons).
- **Write function to compute average period from N sequential interval measurements.**
- Test in software using arbitrary data.
- **Write function to convert period to temperature.**
- Math is pretty easy, can check output against by-hand calculations.

$$\tau = \left(10\ \mu s\ K^{-1}\right) T$$